TonCrown migration — what was done

Completed 31 August 2026. All 106 users, 87 referral-matrix links and 11 stakes moved from the original contract to a fixed, upgradeable one. Verified record by record.

Addresses

	Address
Old contract (retired)	`EQBbOI_xmmkRxsXoutCIlE9JRZsioapGUIwQd88fztMm7-zO`
New contract (live)	`EQBHG-l-XAsYnMGMx8ecP7fU_AM8oSV-r6ZQtYBD4sEGVdPy`
Owner wallet (unchanged)	`EQAHJC-8Mv1xzSEGmDUNde0IqgPJy9JABNtmC_aPP8L9T-hm`

Snapshot taken at mainnet block **89623837**; verification pinned at block **89628943**.

Why it was needed

The original contract handed buyers their payment back. `UpgradeLevel` and `StakeTON` ended with Tact's `self.reply()`, which sends with `SendRemainingValue` (mode 64). Mode 64 attaches the inbound message value, and that counter is not reduced by payouts made earlier in the same transaction — those come out of the account balance. So after distributing 100% of the level price to creators, referrers and spillover, the reply sent the buyer their whole payment back. It only fitted when the contract balance could cover it, and `SendIgnoreErrors` swallowed the failure silently when it could not — which is why it appeared only once the contract was funded.

Measured: a buyer paid **0.0286 TON net** for a 1.25 TON level, and the treasury *fell* 0.0128 TON on the sale.

The original contract had no `SETCODE` receiver, so its code was immutable. Fixing it required a new contract at a new address, and therefore this migration.

Bugs fixed

#	Bug	Effect
1	`self.reply()` after spending inbound value	Refunded the buyer's entire payment
2	Referrer read from every message	Anyone could redirect the 15–25% referral cut; self-referral paid you 50% back forever
3	Matured stake closed without paying	Reward and capital checked separately, both `SendIgnoreErrors` — **principal destroyed**
4	`myBalance()` counted the caller's gas	Every affordability check overstated the treasury
5	Unbounded spillover scan	`exit -14` at ~200 users — registration dies permanently. Contract was at 106
6	Failed USDT deposit stranded jettons	Malformed payload or non-VIP sender lost their USDT
7	`isPaused` could never be set	No emergency stop
8	`getTreasuryLiabilities` gas ceiling	Admin getter died at page 50; found during the testnet rehearsal

What changed for users

Nothing they must do. Levels, VIP class, referral matrix, stakes, accrued check-in balances and earnings totals all carried over.

One thing is reconstructed rather than copied: the original contract never stored the

link referrer separately from the matrix parent, so it is unrecoverable. Migrated users have `linkReferrer = referrer`, meaning their upline takes the combined referral payment (50% at level 1, 30% at level 2+). From their own 7th referral onward the split behaves normally: 15% to the link referrer, 15% to the placement parent.

Spillover is now a constant-gas registry rather than a scan: **117k gas at 1000 users**, against a 1,000,000 ceiling. The old contract used ~4,650 gas per registered user.

Verification

```
users verified:        106
claimable right now:   8.5294 TON
active staked TON:     222.4 TON
All records match.
```

Old and new agree on every platform statistic:

	Old	New
users	106	106
stakedTon	222.4	222.4
activeStakes	11	11
totalDistributed	8.05	8.05

The exported snapshot's summed active stakes equalled `totalStakedTon` exactly, meaning bug #3 never fired on the live contract — no user principal was lost.

Cost

Owner wallet before	19.03 TON
Owner wallet after	8.61 TON
Spent	10.42 TON
Of which landed in the new contract	9.59 TON
Actual fees burned	**0.83 TON**

Incidents

One. At user 35 of 106 the wallet rejected a message with **exit code 133** (seqno mismatch): toncenter was slow, blueprint's internal retry resent against a stale seqno, and the W5 wallet refused it. The message never reached the contract.

Nothing was lost. Progress is recorded only after a send returns, so the item stayed queued, and `RECONCILE=1` re-read all 35 recorded users and confirmed each was really on-chain before resuming. Sends now retry six times with backoff and the settle time went from 2.5s to 4s; two further transient failures after that were absorbed silently.

Still to do

1. **Fund the new contract with 222.4 TON** to cover stake principal. It holds 9.59 TON; 8.53 TON is claimable right now.
2. **Point the frontend at the new address**, then unfreeze.
3. `OwnerWithdraw` the old contract's 0.95 TON.
4. `LockImports` — see below. Best done after real users have confirmed their balances look right, since it cannot be undone.

`SetDistributor` is not needed: `distributorAddress` defaults to the owner address at deployment, so `DistributeDailyRewards` already works from the owner wallet.

What LockImports does

It sets `importsLocked = true`, permanently blocking `ImportUser`, `ImportDownline`,

`ImportStake` and `ImportPlatformTotals`. There is no unlock receiver.

While imports are open the owner can write user records directly — including

`pendingCheckInRewards`, which is claimable TON created from nothing, and stakes of any size that entitle an address to ROI and a principal refund. Locking removes that ability and makes it publicly visible via `getTreasuryStatus.importsLocked` that it is gone.

It does not affect normal operation: registration, upgrades, staking, check-ins and claims behave identically either way.

The trade-off is that it cannot be reversed. If a record later turns out to be missing or wrong, the only recourse is `DEPLOY_NONCE=1` and repeating the whole migration at a new address. Verification passed on all 106 users, so waiting only buys the one check automation cannot perform: a user recognising their own numbers.

`UpgradeContract` could ship code that re-opens imports, so this is not an absolute guarantee against a determined owner — but it is a real lock in the current code, and changing it would be an on-chain code change anyone can see.

Do not unfreeze the frontend while it still points at the old contract: its balance (0.95 TON) is near the ~1.25 TON refund threshold, and it re-arms on its own as registrations accumulate.

Future upgrades need no migration

The new contract has an owner-gated `UpgradeContract` receiver carrying `SETCODE`. Fixes ship to the **same address** with users, stakes and balances untouched — proven in

`tests/Upgradeability.spec.ts`.

The one constraint: `SETCODE` swaps logic, not storage. Changing logic, constants, percentages, receivers and getters is safe. Adding, removing or reordering **storage fields** needs the new code written to read the data already there.

If an import ever needs redoing, `DEPLOY_NONCE=1` gives a clean instance at a new address.

Known limits

- **Stake capital still forwards to** `creatorWallet3` (`forwardStakeCapital = true`, kept at the owner's direction). The contract owes 222.4 TON it does not hold, so solvency depends on funding. `SetStakeCapitalPolicy{false}` would make refunds self-funding.
- **Admin getters have gas ceilings**, measured on real data: `getTreasuryLiabilities` max page 25 (use 20), `getAdminUserSnapshotsPaginated` max 20 (use 10), `getUserListPaginated` fine at 50. These shrink as stakes accumulate.
- **The owner can drain everything** — `OwnerWithdraw`, `UpgradeContract`, `ChangeOwner`.
- **No independent audit.** One reviewer found 7 bugs in code that ran live for months; that is the base rate for a single pass.

Quote

Fixed price for the work described in this report. Quoted in TON.

#	Item	TON
1	Diagnosis — root cause of the refund bug, reproduced in a sandbox and proven against the deployed code hash	5
2	Contract audit — 7 further bugs found and fixed, including one that destroyed user principal and one that would have stopped all registration at ~200 users	12
3	Constant-gas spillover redesign, plus owner-gated `UpgradeContract` so no future fix needs a migration	6
4	Migration tooling — `preflight`, `exportState`, `importState`, `verifyMigration`, `identifyDeployed`, with resume, reconcile and a deployment-nonce escape hatch	7
5	Test suite of 35 tests, plus a full testnet rehearsal using the real 106-user snapshot	5
6	Mainnet migration executed and verified record by record, with documentation and handover	5
	Total	**40**

Notes:

- The 10.42 TON of on-chain costs already spent on the migration is separate and is not included above. 9.59 TON of that is sitting in the new contract's balance, not spent.
- Items 1 and 2 are the substance of the engagement: without them the platform was refunding buyers whenever it held a balance, and was roughly 90 registrations away from a permanent, unfixable halt on the immutable contract.
- Item 3 is why this migration is the last one. Every future fix is a single owner message to the same address.
- Not included: independent third-party audit, frontend changes, or ongoing support.